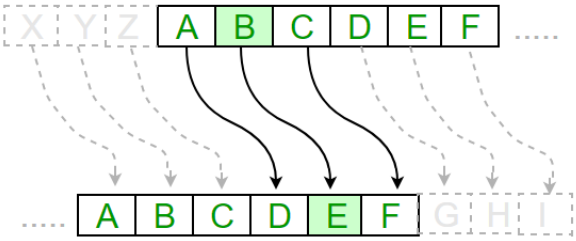
Building a Cipher for Coding and Decoding Messages in Python

Objectives

1. Introduce basic concepts of encoding and decoding messages.
2. Practice using Python skills, including loops, conditionals, and functions.
3. Learn about simple ciphers, specifically the **Caesar Cipher.**
4. Provide opportunities for students new to coding to follow structured steps and for more experienced students to build on the code by adding features.



1. Lesson Introduction: What is a Cipher?

A **cipher** is a way of changing information (like a message) so that it becomes secret. In this lesson, we'll create programs to both **encode** (make secret) and **decode** (make readable) messages using Python. We'll start with a simple cipher, the **Caesar Cipher**.

Caesar Cipher Concept: Each letter in the message is shifted a certain number of places in the alphabet. For example, if we shift each letter by 1:

- "A" becomes "B"
- "B" becomes "C"
- "Z" becomes "A" (the alphabet wraps around)

2. Reviewing Concepts: Important Skills

Before we dive in, let's quickly review some Python concepts that are essential for this project.

a. Variables and strings

Variables help us store information like text and numbers.

```
message = "HELLO WORLD"  # This is our message
shift = 3                # This is how much we'll shift each letter
```

b. Loops and Conditionals

Loops let us repeat code, and conditionals let us check for certain conditions.

```
# Looping through each letter in a string
for letter in message:
    if letter.isalpha():  # Check if it's a letter
        print(f"{letter} is a letter!")
```

c. New Concept: Indexing (strings)

It's important to understand how to work with individual characters in a string. This is where **indexing** comes in!

Indexing is the way we access specific positions, or "indexes," in a string. Each character in a string has a unique position, starting with 0 for the first character, 1 for the second, and so on. Here's a simple example:

```
word = "HELLO"
print(word[0])  # This will print "H"
print(word[1])  # This will print "E"
print(word[2])  # This will print "L"
print(word[4])  # This will print "O"
```

How It Works:

- The square brackets `[ ]` after a string are used to access a specific **index**.
- The number inside the brackets tells Python which character to get from the string.
- Since indexing starts at **0**, `word[0]` gives us the first character (`"H"` in this case).

Using Indexes with Loops

When looping through each character in a string, we don't always need to know its index. But sometimes, knowing the position of a character is useful, especially for encoding ciphers.

Example: Finding the Index of a Letter

Let's say we want to find the index of a specific letter within the alphabet, which we'll use to shift it for a cipher. We can use the `.index()` method for this:

```
alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
letter = "E"
print(alphabet.index(letter))  # This will print 4
```

In this example:

- We use `alphabet.index(letter)` to find where `letter` appears in `alphabet`.
- This is very useful for ciphers, as it tells us how far a letter is from the start of the alphabet.

Negative Indexing

Python also supports **negative indexing**. When we use a negative index, Python counts from the end of the string:

- `-1` is the last character.
- `-2` is the second-to-last character, and so on.

```
word = "HELLO"
print(word[-1])  # This will print "O"
print(word[-2])  # This will print "L"
```

Why This Matters for Ciphers:

- **Accessing specific positions** allows us to find and shift letters easily.
- **Knowing string positions** helps us apply transformations like reversing the alphabet or shifting each character.

Indexing will come in handy as we build ciphers, allowing us to shift letters and create encoded messages!

d. Functions

Functions let us reuse code. We'll use functions to keep our code organized.

```
def encode_caesar(message, shift):
    # Function to encode a message with a Caesar Cipher
    # Code for encoding will go here
    return encoded_message
```

3. Part 1: Coding the Caesar Cipher

For the Caesar Cipher, we can use lists of letters and treat each letter's position in the alphabet as its "index." This approach lets us work with characters directly.

Step 1: Create a list of letters

Let's start by creating a list of all the letters in the alphabet. This will help us find a letter's position, shift it, and get the new letter after the shift.

```
alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
```

We'll use this `alphabet` string to look up and shift letters.

Step 2: Shifting a single letter

To shift a letter, we can:

1. Find its position in the alphabet.
2. Add or subtract the shift amount to get the new position.
3. Use the new position to find the shifted letter in the alphabet.

Here's how that looks in code:

```
def shift_letter(letter, shift):
    # Check if the letter is uppercase and in the alphabet
    if letter in alphabet:
        old_index = alphabet.index(letter)         # Find the current position
        new_index = (old_index + shift) % 26       # Shift and wrap around
        return alphabet[new_index]                 # Get the shifted letter
    else:
        return letter  # Non-letter characters stay the same
```

Explanation of each line:

1. **`if letter in alphabet:`**
   This checks if the letter is in our alphabet list (i.e., it's a letter from A to Z). If it's a special character, space, or number, we won't change it.
2. **`old_index = alphabet.index(letter)`**
   This finds the current position (index) of the letter in the alphabet. For example, if `letter` is `"C"`, `old_index` would be `2` (since "A" is position 0, "B" is 1, and "C" is 2).
3. **`new_index = (old_index + shift) % 26`**
   Here we add the `shift` value to the `old_index` to get the new position of the letter. The `% 26` part wraps the index back around to the beginning if it goes past the end of the alphabet (e.g., "Z" shifted by 1 wraps around to "A"). This is called **modular arithmetic** and keeps the shifting within the bounds of the alphabet.
4. **`return alphabet[new_index]`**
   This line gets the new letter at the `new_index` position in the alphabet and returns it.
5. **`else: return letter`**
   If `letter` is not in the alphabet (for example, a space or punctuation), we return it unchanged.

Step 3: Encoding a Full Message

Now that we can shift a single letter, we'll use this function to shift every letter in a message.

```
def encode_caesar(message, shift):
    encoded_message = ""              # Start with an empty string to build our encoded message
    for letter in message:            # Loop through each letter in the message
        encoded_message += shift_letter(letter, shift)  # Shift the letter and add it to the result
    return encoded_message
```

Explanation of each line:

1. **`encoded_message = ""`**
   We start with an empty string called `encoded_message`, which will hold our final encoded message.
2. **`for letter in message:`**
   This loop goes through each letter in the message one at a time.
3. **`encoded_message += shift_letter(letter, shift)`**
   For each `letter`, we call `shift_letter(letter, shift)` to get the shifted version of that letter, then add it to `encoded_message`. This gradually builds our encoded message as the loop goes through each letter.
4. **`return encoded_message`**
   After the loop finishes, we return the full encoded message.

An example of how you run this code would be: `encoded_caesar("HELLO", 1)`. This shifts the word "hello" by one letter. Since we defined out alphabet using only capital letters, we can only use capital letters here.

4. Part 2: Decoding the Caesar Cipher

To decode a Caesar Cipher, we need to shift each letter in the opposite direction, which we can do by putting a negative sign in front of the shift.

```
def decode_caesar(encoded_message, shift):
    return encode_caesar(encoded_message, -shift)
```

Example usage:

```
decoded_message = decode_caesar(encoded_message, shift)
print(f"Decoded Message: {decoded_message}")
```

5. Advanced Option: Adding More Cipher Features

For students looking for more of a challenge, here are some additional features you can add to their Caesar Cipher. Each feature builds on what you've already done, using the `alphabet` string and basic Python skills.

- Option 1: Allow lowercase letters
  - Hint: define both uppercase and lowercase alphabets, then check to see if the letter is in the uppercase or the lowercase alphabet.
- Option 2: Change the shift amount
  - Instead of using a fixed shift, try asking for user input using `input()` or generate a random shift each time using the `random` package.
    - ex. `random.randint(1, 25)` selects a random number between 1 and 25.
- Option 3: Change the alphabet or add symbols
  - Add extra characters or symbols to your alphabet variable to include them in the cipher!
- Option 4: Shift vowels and consonants separately
  - Use different shift amounts for vowels and consonants to create a more complex cipher.
    - Hint: use `if-else` statements!
- Option 5: Make a "mirror" cipher
  - Add a rule that mirrors letters around the alphabet (e.g. A becomes Z, B becomes Y, etc.).
- Option 6: Skip every other letter
  - Create a cipher that only shifts every other letter in the message, leaving the rest as they are.